

JFLAP LABORATORY ASSIGNMENT

Applications of Turing Machines in Addition

Laboratory Platform: JFLAP

Focus: Unary, Binary, Decimal and Multi-Tape Addition

Prepared for practical implementation and group demonstration

1. Laboratory Overview

This laboratory assignment gives 21 groups different addition problems to implement as Turing Machines in JFLAP. The problems are based on Kenyan contexts and progress from basic unary addition to binary, decimal, multi-operand, input-validation and multi-tape designs.

2. General Representation Conventions

- Unary representation: each symbol 1 represents one unit.
- Binary representation: numbers use 0 and 1.
- The symbol # separates operands unless another separator is specified.
- B represents the blank tape symbol.
- Recommended state names are q_0 for the start state, q_{accept} for acceptance and q_{reject} for rejection.
- For monetary examples, one unary symbol may represent a specified amount such as KSh 100.

Basic unary example: $4 + 3 = 7$

```
1111#111  →  1111111
```

3. Submission Requirements for Every Group

1. A brief explanation of the Kenyan application and the addition problem.
2. The formal definition of the Turing Machine: states, input alphabet, tape alphabet, transition function, start state, accept state and reject state.
3. The completed JFLAP .jff file.
4. A clearly labelled state-transition diagram.
5. A short explanation of the purpose of every state.
6. A complete computation trace for the assigned input.
7. At least three additional valid test cases.
8. At least one boundary, edge or invalid test case.
9. Screenshots showing the machine running and accepting correct inputs.
10. A brief discussion of limitations and one suggested improvement.

Group 1: M-Pesa Deposit Addition

An M-Pesa agent in Nyeri receives two deposits. Each unary symbol 1 represents KSh 100. The first customer deposits KSh 400 and the second deposits KSh 300.

Input

```
1111#111
```

Expected output

```
1111111
```

Task

Design a single-tape Turing Machine that removes the separator and produces one unary string representing the total deposit.

Group 2: Milk Collection at a Dairy Cooperative

A dairy cooperative in Nyandarua receives milk from two farmers. Each 1 represents one 10-litre container. The cooperative must calculate the total while retaining both original records.

Input

```
111#11
```

Expected output

```
111#11#11111
```

Task

Design a Turing Machine that preserves both original numbers and writes their sum after a second separator.

Group 3: Matatu Passenger Counting

A matatu operating between Nyeri and Karatina picks up two passengers at the first stage, three at the second stage and one at the third stage.

Input

```
11#111#1
```

Expected output

```
111111
```

Task

Design a Turing Machine that adds three unary numbers and produces one unary result.

Group 4: Adding Zero Mobile-Money Transactions

A mobile-money agent sometimes records a period in which one service point receives no deposit. Zero is represented by an empty unary string.

Input(s)

```
1111#  
#111
```

Expected output(s)

```
1111  
111
```

Task

Design a Turing Machine that adds two unary numbers when either operand may be zero.

- The machine must accept both examples.
- The machine must also process # as $0 + 0$ and return an empty unary result.

Group 5: School Meal Registration Incrementer

A public primary school in Laikipia records learners who have received lunch. Each 1 represents one learner. When another learner is served, the stored number must increase by one.

Input

11111

Expected output

111111

Task

Design a Turing Machine that adds one to any valid unary number.

- An empty tape represents zero and must produce 1.
- The machine must halt after writing exactly one additional symbol.

Group 6: Adding a Fixed Number of Fertilizer Bags

A farmer in Trans Nzoia records fertilizer bags already in storage. Every new delivery adds exactly three bags.

Input

1111

Expected output

1111111

Task

Design a Turing Machine that adds the constant unary value 111 to any valid unary input.

Required test cases

B → 111
1 → 1111
1111 → 1111111
1111111111 → 1111111111111

Group 7: Water Delivery from Multiple Suppliers

A school in Kitui receives water from several suppliers. Each 1 represents one 1,000-litre tank. The number of suppliers can vary from one input to another.

Input

```
11#1#111#11
```

Expected output

```
11111111
```

Task

Design a Turing Machine that adds any number of unary operands separated by #.

- The design must not be limited to exactly four operands.
- The final tape must contain a single unary number with no separators.

Group 8: Election Vote-Tally Addition

Two polling streams at a Kenyan polling station report votes for the same candidate. Each vote is represented by 1.

Input

```
11111#111
```

Expected output

```
11111111
```

Special requirement: Use temporary symbols such as X and Y to mark processed votes.

Task

Design a marker-based Turing Machine that transfers votes from the second stream to the first stream.

- Mark one vote from the second stream.
- Transfer or reproduce it in the first stream.
- Repeat until all votes have been processed.
- Remove every temporary symbol before accepting.

Group 9: Kenya Power Meter Reading Addition

A prepaid electricity system stores small meter values in binary. The two readings below represent 5 and 2.

Input

0101#0010

Expected output

111

Task

Design a Turing Machine that adds two equal-length binary numbers.

- The result must be binary.
 - Remove unnecessary leading zeros before acceptance.
-

Group 10: Binary Addition with a Final Carry

A digital payment terminal records two binary transaction values. The input below represents $7 + 1$.

Input

111#1

Expected output

1000

Task

Design a binary addition Turing Machine that creates a new leftmost bit when a final carry remains after the most significant digits have been processed.

Group 11: Binary Numbers of Different Lengths

A county revenue system stores daily collections in binary. The input below represents $21 + 7$.

Input

10101#111

Expected output

11100

Task

Design a Turing Machine that adds binary numbers of different lengths.

- Treat missing leading positions as zero.
- Test both cases where the first operand is longer and where the second operand is longer.

Group 12: Binary Addition with Repeated Carries

A Kenyan supermarket adds two binary sales values. The input represents $15 + 15$ and requires several consecutive carry operations.

Input

1111#1111

Expected output

11110

Task

Design a Turing Machine that correctly handles a chain of repeated carries.

- Use distinguishable states for addition without carry.
- Use separate states for addition with carry.
- Use a state for processing any final carry.

Group 13: Binary Incrementer for Vehicle Counting

A digital traffic counter on the Nairobi-Nakuru highway stores the number of vehicles in binary. One more vehicle passes through the checkpoint.

Input

10111

Expected output

11000

Task

Design a Turing Machine that adds binary 1 to any valid binary input.

Required test cases

0 → 1
1 → 10
1010 → 1011
1111 → 10000

Group 14: Adding a Fixed Binary Value

A county health office receives two additional vaccine boxes every morning. The current stock is stored in binary. The input below represents 21 boxes, and the machine must add binary 10.

Input

10101

Expected output

10111

Task

Design a Turing Machine that adds the fixed binary value 10 to any valid binary number.

- The input contains only the current stock; the constant 10 is built into the machine.
- Include test cases that require no carry and test cases that require a carry.

Group 15: Preserving Binary Transaction Records

A bank system must retain two original binary transactions and append their total. The input represents $5 + 3$.

Input

101#11

Expected output

101#11#1000

Task

Design a Turing Machine that copies the operands, performs binary addition and writes the result after a second separator.

- The original operands must remain unchanged.
- All temporary markers must be removed before acceptance.

Group 16: Adding Three Binary Revenue Values

Three county offices submit daily revenue totals in binary. The input represents $5 + 3 + 2 = 10$.

Input

101#11#10

Expected output

1010

Task

Design a Turing Machine that adds three binary numbers.

- The machine may first add the first two operands.
- It may then add the third operand to the intermediate result.
- The final tape must contain only the final binary sum.

Group 17: Multi-Tape M-Pesa Transaction Addition

An M-Pesa system receives two binary values from separate transaction sources. Tape 1 contains 1011 (11), Tape 2 contains 110 (6), and Tape 3 is initially blank.

Input(s)

Tape 1: 1011
Tape 2: 110
Tape 3: B

Expected output

Tape 3: 10001

Task

Using JFLAP's multi-tape Turing Machine feature, design a three-tape machine that places the sum on Tape 3.

- Tape 1 stores the first operand.
- Tape 2 stores the second operand.
- Tape 3 stores the answer.
- The original operands must remain unchanged.

Group 18: Single-Digit Decimal Market Sales Addition

A trader at Karatina Market records fruit crates sold by two attendants using decimal digits.

Input

4#3

Expected output

7

Task

Design a Turing Machine that adds two single decimal digits when their total is less than ten.

- Support valid combinations from 0#0 to 9#0 as long as the answer remains one digit.
- Reject malformed inputs such as 12#3 or 4##3.

Group 19: Decimal Addition Producing a Carry

A shop in Nairobi sells eight bags of rice in the morning and seven bags in the afternoon. The machine must produce a two-digit decimal result.

Input

8#7

Expected output

15

Task

Design a Turing Machine that adds two single decimal digits and produces a two-digit result when the sum is ten or greater.

Required test cases

6#7 → 13
9#9 → 18
5#5 → 10
8#4 → 12

Group 20: Validating Unary Addition Inputs

A county data-entry system must reject incorrectly entered addition expressions before attempting the calculation.

Input

111#11

Expected output

11111

Task

Design a Turing Machine that first validates the input and then performs unary addition.

- Accept only strings of the form $1^m\#1^n$, where $m, n \geq 0$.
- Reject every input containing an invalid symbol or an incorrect number of separators.

Required test cases

11A#111 → reject
11##111 → reject
→ accept as $0 + 0$
111#11#1 → reject
101#11 → reject

Group 21: Maize Collection with 0 as the Separator

A maize cooperative in Uasin Gishu records bags collected from two farms. Each 1 represents one bag, while 0 separates the two unary numbers. The input contains four bags from the first farm and three from the second.

Input

```
11110111
```

Expected output

```
1111111
```

Task

Design a Turing Machine that adds two unary numbers represented in the form 1^m01^n .

- Confirm that exactly one 0 separator is present.
 - Remove the separator while adding the operands.
 - Reject any input containing more than one 0.
 - Test cases must include an empty left operand and an empty right operand.
-

4. Assessment Rubric

Assessment area	Marks
Understanding of the Kenyan case	5
Correct input and output specification	5
State-transition design	20
Correct JFLAP implementation	25
Testing using different inputs	15
Complete computation trace	10
Handling edge and invalid cases	10
Explanation and presentation	10
Total	100

5. Laboratory Demonstration Requirements

1. Explain the Kenyan problem being solved.
2. Open and run the completed JFLAP machine.
3. Demonstrate the assigned input.
4. Demonstrate at least one additional valid input.
5. Demonstrate one edge case or invalid input.
6. Explain the purpose of every state.
7. Show how the tape changes during execution.
8. Explain whether the machine always halts.
9. State the main limitation of the design.
10. Suggest one realistic improvement or extension.

6. Lecturer's Note

The problems are deliberately different. Groups should not submit the same machine with only different input values. The implementation must reflect the unique requirement assigned to the group, such as preserving operands, handling zero, using marker symbols, supporting multiple operands, validating input, using binary or decimal notation, or operating on multiple tapes.